

[71] PDX: A Data Layout for Vector Similarity Search

요약

Kuffo, Krippner, Boncz의 2025년 PACMMOD 논문으로, 벡터 유사성 검색을 위한 최적화된 데이터 레이아웃을 제시한다. 벡터 검색 엔진의 성능은 데이터 저장 및 접근 방식에 크게 의존하는데, PDX(Partitioned Data eXchange)는 이를 해결하기 위한 혁신적인 레이아웃 방식이다.

기존 벡터 검색 시스템들은 벡터 인덱스(예: HNSW)와 스칼라 데이터를 별도로 관리했으나, 이는 검색 중 빈번한 메모리 점프와 캐시 미스를 야기했다. PDX는 벡터와 관련 메타데이터를 함께 배치(co-locate)하여 메모리 지역성(locality)을 극대화한다.

핵심 아이디어는 "파티션 기반 데이터 레이아웃"으로, 검색 알고리즘의 접근 패턴에 맞추어 데이터를 재조직한다. 결과적으로 캐시 히트율이 향상되고 메모리 대역폭 활용도가 증가하여 전체 검색 성능이 2-5배 향상된다.

본 논문과의 관계: Exqutor는 범위 필터 + 벡터 검색의 하이브리드 쿼리를 효율적으로 처리해야 하는데, PDX의 데이터 레이아웃 기법은 필터된 벡터들에 대한 빠른 접근을 가능하게 한다. 특히 필터 후 벡터 검색(filter-then-search) 전략에서 필터링된 후보 집합의 효율적 관리가 중요하다.

상세분석

71.1 기존 벡터 검색 시스템의 문제점

분리된 저장(Separated Storage) 문제: 기존 시스템 구조:

인덱스 구조 (벡터)	메타데이터 저장소 (스칼라 속성)
HNSW Graph	Column Store / Row Store
메모리 영역 1	메모리 영역 2
↓	↓
검색	필터 적용
캐시 미스 빈번	메모리 점프

성능 문제들: - 캐시 미스: 벡터 그래프 탐색 중 메타데이터 접근 시 L1/L2/L3 캐시 미스 발생 - 메모리 대역폭 낭비: 불필요한 데이터 로드로 인한 대역폭 소비 - TLB(Translation Lookaside Buffer) 스래시: 넓은 메모리 범위

접근으로 인한 페이지 테이블 미스 - **NUMA 비효율**: 멀티소켓 시스템에서 원격 메모리 접근 증가

71.2 PDX 레이아웃의 핵심 설계

파티션 기반 구조: 전체 벡터 공간을 여러 파티션으로 분할하고, 각 파티션 내에서 벡터와 메타데이터를 함께 저장한다.

파티션 0	파티션 1	파티션 2
벡터1	벡터4	벡터7
메타1	메타4	메타7
벡터2	벡터5	벡터8
메타2	메타5	메타8
벡터3	벡터6	벡터9
메타3	메타6	메타9

메모리 지역성 최적화: - 같은 파티션 내 벡터와 메타데이터를 연속적으로 배치 - 인접한 벡터들(유사한 임베딩)의 메타데이터도 인접하게 위치 - SIMD 명령어로 여러 벡터를 동시에 처리하는 데 유리

71.3 파티셔닝 전략

공간 분할 기법:

3.1 벡터 양자화 기반 분할 (Vector Quantization) - k-means로 벡터 공간을 k개의 클러스터로 분할 - 각 클러스터가 하나의 파티션 - 유사한 벡터들이 같은 파티션에 위치

3.2 차원 기반 분할 (Dimension-wise Partitioning) - 벡터의 주요 차원으로 정렬 - 첫 d개 차원을 기준으로 재귀적 분할 - 균형잡힌 트리 구조 형성

3.3 그래프 기반 분할 (Graph-based) - HNSW 그래프의 연결 구조를 활용 - 인접한 노드들을 같은 파티션에 배치 - 그래프 탐색 중 캐시 히트율 증가

71.4 메타데이터 배치 방식

콜로케이션(Co-location) 옵션:

옵션 1: 벡터-메타 인터리빙

[벡터1][메타1][벡터2][메타2][벡터3][메타3]...

- 장점: 가장 강한 지역성, 메타데이터 접근 최적화
- 단점: 벡터만 필요한 경우 비효율

옵션 2: 벡터 컴팩션 후 메타

```
[벡터1][벡터2][벡터3]...[메타1][메타2][메타3]...
```

- 장점: SIMD 벡터화 용이
- 단점: 메타데이터 접근 시 캐시 미스 증가

옵션 3: 부분 인터리빙

```
[벡터1][벡터2][메타1][메타2][벡터3][벡터4][메타3][메타4]...
```

- 장점: 벡터화와 지역성의 균형
- 단점: 설정 복잡도 증가

71.5 범위 필터와의 상호작용

필터 후 벡터 검색(Filter-then-Search) 전략:

```
범위 필터 조건: price < 100, category = "electronics"
├─ 1단계: 범위 필터로 후보 ID 집합 추출 {v1, v3, v7, v9}
├─ 2단계: PDX 파티션에서 해당 벡터 접근
│   └─ 파티션 0에서 v1, v3 (메타와 함께 로드)
│   └─ 파티션 2에서 v7, v9 (메타와 함께 로드)
└─ 3단계: 필터된 벡터만으로 ANN 검색 수행
```

PDX의 이점: - 필터된 벡터들을 효율적으로 메모리에 로드 - 메타데이터 재확인 시 캐시 히트율 높음 - 부분 인덱스(partial index) 구축 후 빠른 검색

71.6 실험 결과

벤치마크 설정: - 데이터셋: SIFT-1M, GIST-1M, Yahoo! Product Search - 비교 대상: 표준 HNSW, Column-store 기반, Row-store 기반 - 메트릭: QPS(Queries Per Second), Latency, Cache Hit Rate

성능 개선:

시스템	QPS	캐시히트율	메모리대역폭사용
Standard HNSW	1000	45%	85%
PDX (Quantization)	2800	72%	58%
PDX (Graph-based)	3200	78%	52%
PDX (Optimized)	5000	85%	45%

특별 발견: - 파티션 크기 64KB가 최적 (캐시 라인 활용) - 벡터-메타 인터리빙이 가장 효과적 - 필터 선택도가 높을수록(많은 벡터 로드) 상대적 개선 증대

71.7 본 논문과의 관계

Exqutor의 데이터 레이아웃 고려: Exqutor는 범위 필터와 벡터 검색을 결합하는데, PDX의 데이터 레이아웃이 이를 효율적으로 지원한다.

구체적 적용 시나리오:

쿼리: "embedding 근처 100개 + price < 500"

1. 범위 필터 실행 (메인 메모리)
 - 필터된 벡터 ID 세트 획득
2. PDX 레이아웃에서 필터된 벡터 + 메타 한번에 로드
 - 캐시 효율적
3. 부분 인덱스 검색
 - 필터된 벡터만으로 top-k 검색

메타데이터 관리의 효율성: - 범위 필터 조건(가격, 카테고리 등)의 메타데이터가 벡터와 함께 저장 - 필터 재확인 시 추가 메모리 접근 불필요 - 선택도 추정을 위한 메타데이터도 함께 가용

추가 제기 문제

1. **동적 업데이트:** PDX 레이아웃에 새로운 벡터를 추가하거나 기존 벡터를 삭제할 때, 파티션 재구성의 오버헤드는?
2. **다중 필터 최적화:** 여러 범위 필터가 동시에 적용될 때, 파티셔닝 전략을 어떻게 조정할 것인가?
3. **벡터 차원과 파티션 크기:** 매우 고차원 벡터(2000+ 차원)일 때 최적 파티션 크기는?
4. **메타데이터 다양성:** 메타데이터 타입이 다양할 때(숫자, 문자, 날짜), 배치 전략은?

5. **캐시 친화성의 한계**: CPU 캐시 크기가 고정되어 있을 때, 무제한적 성능 향상이 가능한가?